In [6]:
```
----TASK 1--------
# Data Manipulation
import pandas as pd
import numpy as np

# Data Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning (for future tasks)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Display settings (so all columns show properly)
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
```

In [7]:
```
df = pd.read_excel("HousePrediction.xlsx")

print("Dataset Loaded Successfully ✅")
```

```
Dataset Loaded Successfully ✅
```

In [8]:
```
print("Shape of dataset:")
df.shape
```

```
Shape of dataset:
```

Out[8]:
```
(2919, 13)
```

In [9]:
```
print("Columns in Dataset:")
df.columns
```

```
Columns in Dataset:
```

Out[9]:
```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotArea', 'LotConfig', 'BldgType',
       'OverallCond', 'YearBuilt', 'YearRemodAdd', 'Exterior1st', 'BsmtFinSF2',
       'TotalBsmtSF', 'SalePrice'],
      dtype='object')
```

In [10]:
```
df.head(100)
```

Out[10]:

	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
0	0	60	RL	8450	Inside	1Fam	5	2003
1	1	20	RL	9600	FR2	1Fam	8	1976
2	2	60	RL	11250	Inside	1Fam	5	2001
3	3	70	RL	9550	Corner	1Fam	5	1915
4	4	60	RL	14260	FR2	1Fam	5	2000
5	5	50	RL	14115	Inside	1Fam	5	1993
6	6	20	RL	10084	Inside	1Fam	5	2004
7	7	60	RL	10382	Corner	1Fam	6	1973
8	8	50	RM	6120	Inside	1Fam	5	1931
9	9	190	RL	7420	Corner	2fmCon	6	1939
10	10	20	RL	11200	Inside	1Fam	5	1965
11	11	60	RL	11924	Inside	1Fam	5	2005
12	12	20	RL	12968	Inside	1Fam	6	1962
13	13	20	RL	10652	Inside	1Fam	5	2006
14	14	20	RL	10920	Corner	1Fam	5	1960
15	15	45	RM	6120	Corner	1Fam	8	1929
16	16	20	RL	11241	CulDSac	1Fam	7	1970
17	17	90	RL	10791	Inside	Duplex	5	1967
18	18	20	RL	13695	Inside	1Fam	5	2004
19	19	20	RL	7560	Inside	1Fam	6	1958
20	20	60	RL	14215	Corner	1Fam	5	2005
21	21	45	RM	7449	Inside	1Fam	7	1930
22	22	20	RL	9742	Inside	1Fam	5	2002
23	23	120	RM	4224	Inside	TwnhsE	7	1976
24	24	20	RL	8246	Inside	1Fam	8	1968
25	25	20	RL	14230	Corner	1Fam	5	2007
26	26	20	RL	7200	Corner	1Fam	7	1951
27	27	20	RL	11478	Inside	1Fam	5	2007
28	28	20	RL	16321	CulDSac	1Fam	6	1957
29	29	30	RM	6324	Inside	1Fam	6	1927
30	30	70	C (all)	8500	Inside	1Fam	4	1920
31	31	20	RL	8544	CulDSac	1Fam	6	1966
32	32	20	RL	11049	Corner	1Fam	5	2007

	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
33	33	20	RL	10552	Inside	1Fam	5	1959
34	34	120	RL	7313	Inside	TwnhsE	5	2005
35	35	60	RL	13418	Inside	1Fam	5	2004
36	36	20	RL	10859	Corner	1Fam	5	1994
37	37	20	RL	8532	Inside	1Fam	6	1954
38	38	20	RL	7922	Inside	1Fam	7	1953
39	39	90	RL	6040	Inside	Duplex	5	1955
40	40	20	RL	8658	Inside	1Fam	5	1965
41	41	20	RL	16905	Inside	1Fam	6	1959
42	42	85	RL	9180	CulDSac	1Fam	7	1983
43	43	20	RL	9200	CulDSac	1Fam	6	1975
44	44	20	RL	7945	Inside	1Fam	6	1959
45	45	120	RL	7658	Inside	TwnhsE	5	2005
46	46	50	RL	12822	CulDSac	1Fam	5	2003
47	47	20	FV	11096	Inside	1Fam	5	2006
48	48	190	RM	4456	Inside	2fmCon	5	1920
49	49	20	RL	7742	Inside	1Fam	7	1966
50	50	60	RL	13869	Corner	1Fam	6	1997
51	51	50	RM	6240	Inside	1Fam	6	1934
52	52	90	RM	8472	Corner	Duplex	5	1963
53	53	20	RL	50271	Inside	1Fam	5	1981
54	54	80	RL	7134	Inside	1Fam	5	1955
55	55	20	RL	10175	Inside	1Fam	5	1964
56	56	160	FV	2645	Inside	Twnhs	5	1999
57	57	60	RL	11645	Corner	1Fam	5	2004
58	58	60	RL	13682	CulDSac	1Fam	5	2006
59	59	20	RL	7200	Inside	1Fam	7	1972
60	60	20	RL	13072	Inside	1Fam	5	2004
61	61	75	RM	7200	Inside	1Fam	7	1920
62	62	120	RL	6442	Inside	TwnhsE	5	2006
63	63	70	RM	10300	Inside	1Fam	6	1921
64	64	60	RL	9375	Inside	1Fam	5	1997
65	65	60	RL	9591	Inside	1Fam	5	2004

	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
66	66	20	RL	19900	Inside	1Fam	5	1970
67	67	20	RL	10665	Inside	1Fam	5	2003
68	68	30	RM	4608	Corner	1Fam	6	1945
69	69	50	RL	15593	Corner	1Fam	4	1953
70	70	20	RL	13651	Inside	1Fam	6	1973
71	71	20	RL	7599	Corner	1Fam	6	1982
72	72	60	RL	10141	Corner	1Fam	5	1998
73	73	20	RL	10200	Inside	1Fam	7	1954
74	74	50	RM	5790	Corner	1Fam	6	1915
75	75	180	RM	1596	Inside	Twnhs	5	1973
76	76	20	RL	8475	Inside	1Fam	7	1956
77	77	50	RM	8635	Inside	1Fam	5	1948
78	78	90	RL	10778	Inside	Duplex	5	1968
79	79	50	RM	10440	Corner	1Fam	6	1910
80	80	60	RL	13000	Corner	1Fam	6	1968
81	81	120	RM	4500	FR2	TwnhsE	5	1998
82	82	20	RL	10206	Inside	1Fam	5	2007
83	83	20	RL	8892	Inside	1Fam	5	1960
84	84	80	RL	8530	Inside	1Fam	5	1995
85	85	60	RL	16059	Corner	1Fam	5	1991
86	86	60	RL	11911	Inside	1Fam	5	2005
87	87	160	FV	3951	Corner	TwnhsE	5	2009
88	88	50	C (all)	8470	Corner	1Fam	2	1915
89	89	20	RL	8070	Inside	1Fam	5	1994
90	90	20	RL	7200	Inside	1Fam	5	1950
91	91	20	RL	8500	Inside	1Fam	3	1961
92	92	30	RL	13360	Inside	1Fam	7	1921
93	93	190	C (all)	7200	Corner	2fmCon	6	1910
94	94	60	RL	9337	Inside	1Fam	5	1997
95	95	60	RL	9765	Corner	1Fam	8	1993
96	96	20	RL	10264	Inside	1Fam	5	1999
97	97	20	RL	10921	Inside	1Fam	5	1965
98	98	30	RL	10625	Corner	1Fam	5	1920

	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
99	99	20	RL	9320	Inside	1Fam	5	1959

In [11]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2919 entries, 0 to 2918
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   Id            2919 non-null   int64  
 1   MSSubClass    2919 non-null   int64  
 2   MSZoning      2915 non-null   object 
 3   LotArea       2919 non-null   int64  
 4   LotConfig     2919 non-null   object 
 5   BldgType      2919 non-null   object 
 6   OverallCond   2919 non-null   int64  
 7   YearBuilt     2919 non-null   int64  
 8   YearRemodAdd  2919 non-null   int64  
 9   Exterior1st   2918 non-null   object 
 10  BsmtFinSF2    2918 non-null   float64
 11  TotalBsmtSF   2918 non-null   float64
 12  SalePrice     1460 non-null   float64
dtypes: float64(3), int64(6), object(4)
memory usage: 296.6+ KB
```

In [12]:
```
df.describe()
```

Out[12]:

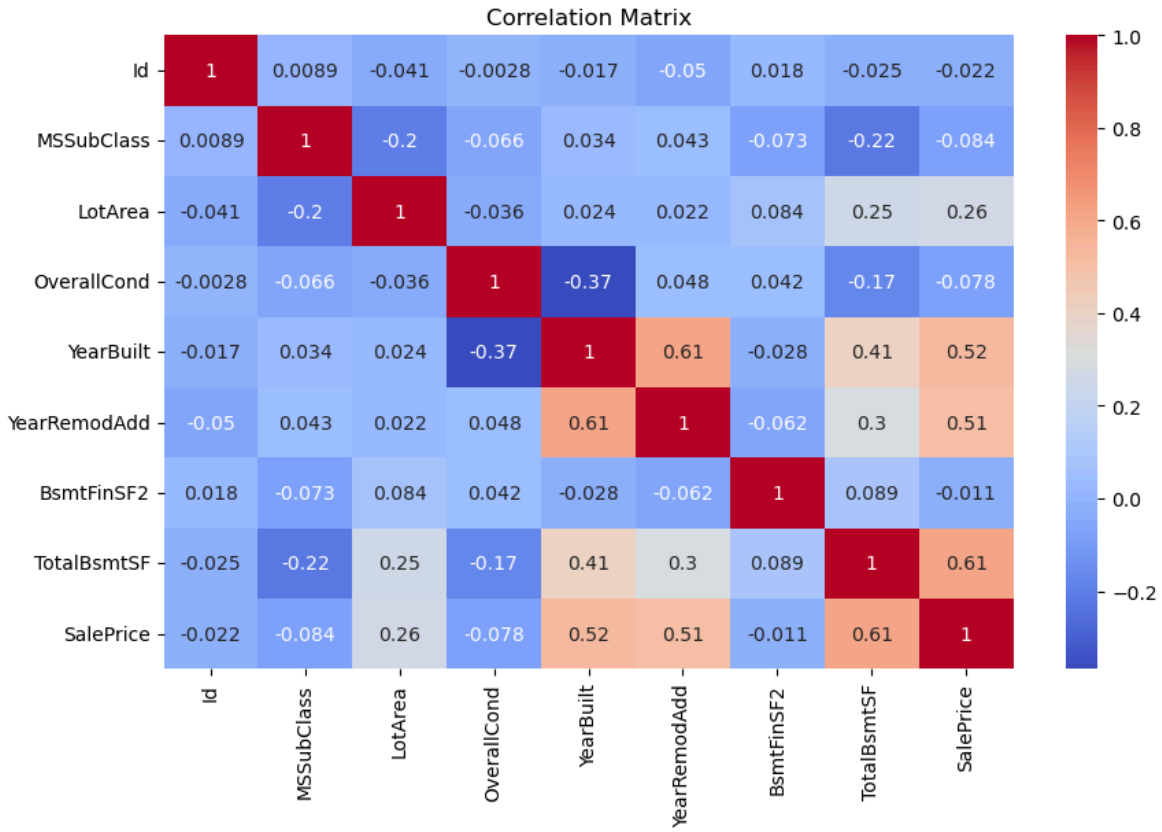
	Id	MSSubClass	LotArea	OverallCond	YearBuilt	YearRemodA
count	2919.000000	2919.000000	2919.000000	2919.000000	2919.000000	2919.000
mean	1459.000000	57.137718	10168.114080	5.564577	1971.312778	1984.264
std	842.787043	42.517628	7886.996359	1.113131	30.291442	20.894
min	0.000000	20.000000	1300.000000	1.000000	1872.000000	1950.000
25%	729.500000	20.000000	7478.000000	5.000000	1953.500000	1965.000
50%	1459.000000	50.000000	9453.000000	5.000000	1973.000000	1993.000
75%	2188.500000	70.000000	11570.000000	6.000000	2001.000000	2004.000
max	2918.000000	190.000000	215245.000000	9.000000	2010.000000	2010.000

In [13]:
```
print("Missing Values in Each Column:")
df.isnull().sum()
```

```
Missing Values in Each Column:
```

Out[13]:
```
Id                 0
MSSubClass         0
MSZoning           4
LotArea            0
LotConfig          0
BldgType           0
OverallCond        0
YearBuilt          0
YearRemodAdd       0
Exterior1st        1
BsmtFinSF2         1
TotalBsmtSF        1
SalePrice       1459
dtype: int64
```

In [14]:
```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



In [15]:
```
# Separate categorical and numerical columns
categorical_cols = df.select_dtypes(include=['object']).columns
numerical_cols = df.select_dtypes(exclude=['object']).columns

print("Categorical Columns:")
print(categorical_cols)

print("\nNumerical Columns:")
print(numerical_cols)
```

```
Categorical Columns:
Index(['MSZoning', 'LotConfig', 'BldgType', 'Exterior1st'], dtype='object')

Numerical Columns:
Index(['Id', 'MSSubClass', 'LotArea', 'OverallCond', 'YearBuilt',
       'YearRemodAdd', 'BsmtFinSF2', 'TotalBsmtSF', 'SalePrice'],
      dtype='object')
```

In [ ]:
```
---------- TASK 2-------------------------
```

In [16]:
```
# Check missing values in each column
missing_values = df.isnull().sum()

print("Missing Values in Each Column:")
print(missing_values)
```

```
Missing Values in Each Column:
Id                 0
MSSubClass         0
MSZoning           4
LotArea            0
LotConfig          0
BldgType           0
OverallCond        0
YearBuilt          0
YearRemodAdd       0
Exterior1st        1
BsmtFinSF2         1
TotalBsmtSF        1
SalePrice       1459
dtype: int64
```

In [17]:
```
# Columns that contain missing values
features_with_nan = [col for col in df.columns if df[col].isnull().sum() > 0]

print("Features with Missing Values:")
for col in features_with_nan:
    print(f"{col} : {df[col].isnull().sum()} missing values")
```

```
Features with Missing Values:
MSZoning : 4 missing values
Exterior1st : 1 missing values
BsmtFinSF2 : 1 missing values
TotalBsmtSF : 1 missing values
SalePrice : 1459 missing values
```

In [18]:
```
# Use only training data
train_df = df[df['SalePrice'].notnull()]

for col in features_with_nan:
    print(f"\nAnalyzing: {col}")
    print(train_df.groupby(train_df[col].isnull())['SalePrice'].mean())
```

```
Analyzing: MSZoning
MSZoning
False    180921.19589
Name: SalePrice, dtype: float64

Analyzing: Exterior1st
Exterior1st
False    180921.19589
Name: SalePrice, dtype: float64

Analyzing: BsmtFinSF2
BsmtFinSF2
False    180921.19589
Name: SalePrice, dtype: float64

Analyzing: TotalBsmtSF
TotalBsmtSF
False    180921.19589
Name: SalePrice, dtype: float64

Analyzing: SalePrice
SalePrice
False    180921.19589
Name: SalePrice, dtype: float64
```

In [19]:
```
numerical_features = df.select_dtypes(exclude=['object']).columns

print("Number of Numerical Features:", len(numerical_features))
print("Numerical Features List:")
print(numerical_features)
```

```
Number of Numerical Features: 9
Numerical Features List:
Index(['Id', 'MSSubClass', 'LotArea', 'OverallCond', 'YearBuilt',
       'YearRemodAdd', 'BsmtFinSF2', 'TotalBsmtSF', 'SalePrice'],
      dtype='object')
```

In [20]:
```
df[numerical_features].head()
```

Out[20]:

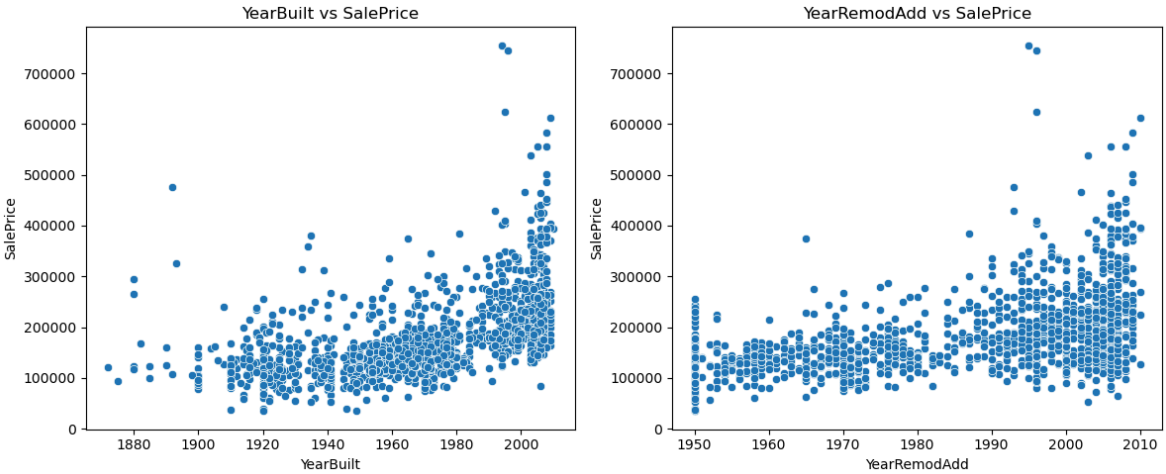
	Id	MSSubClass	LotArea	OverallCond	YearBuilt	YearRemodAdd	BsmtFinSF2	Total
0	0	60	8450	5	2003	2003	0.0	
1	1	20	9600	8	1976	1976	0.0	
2	2	60	11250	5	2001	2002	0.0	
3	3	70	9550	5	1915	1970	0.0	
4	4	60	14260	5	2000	2000	0.0	

In [22]:
```
# Create new features
train_df['HouseAge'] = 2024 - train_df['YearBuilt']
train_df['RemodelAge'] = 2024 - train_df['YearRemodAdd']

# Compare with SalePrice
print(train_df[['YearBuilt', 'YearRemodAdd', 'HouseAge', 'RemodelAge', 'SalePric

plt.figure(figsize=(12,5))
```

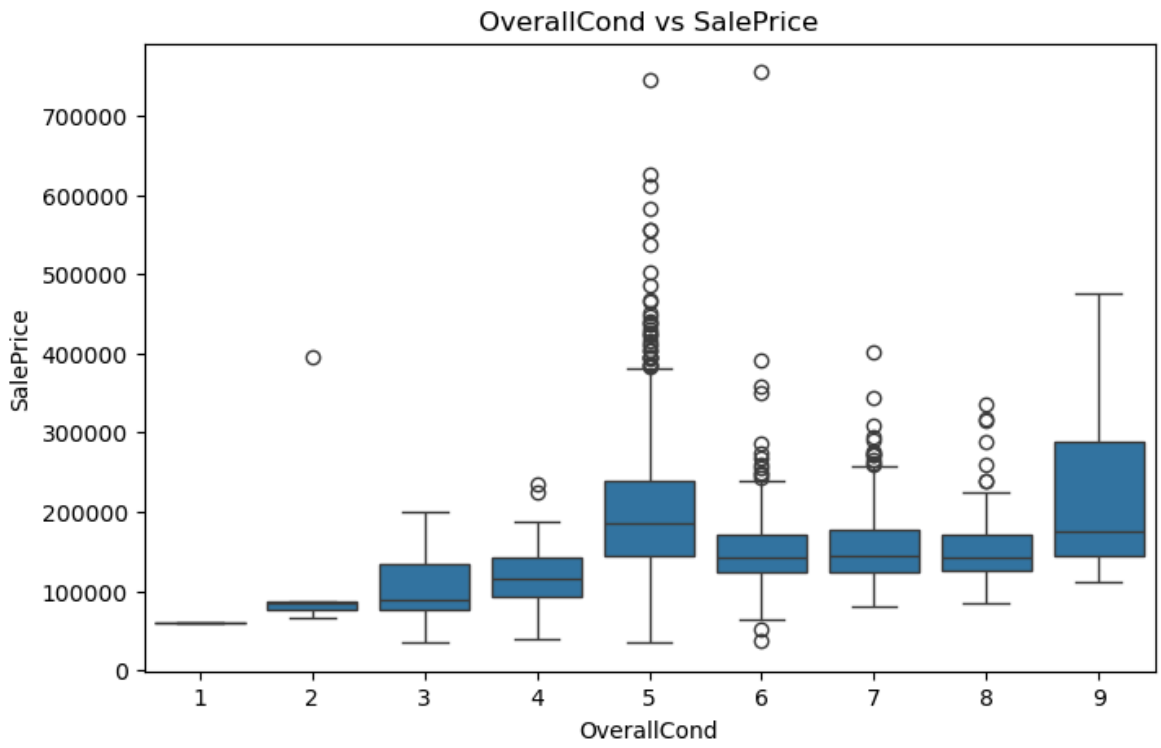
```
plt.subplot(1,2,1)
sns.scatterplot(x=train_df['YearBuilt'], y=train_df['SalePrice'])
plt.title("YearBuilt vs SalePrice")

plt.subplot(1,2,2)
sns.scatterplot(x=train_df['YearRemodAdd'], y=train_df['SalePrice'])
plt.title("YearRemodAdd vs SalePrice")

plt.tight_layout()
plt.show()
```

```
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\243907548.py:2: SettingWithCopy
Warning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  train_df['HouseAge'] = 2024 - train_df['YearBuilt']
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\243907548.py:3: SettingWithCopy
Warning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  train_df['RemodelAge'] = 2024 - train_df['YearRemodAdd']
```

	YearBuilt	YearRemodAdd	HouseAge	RemodelAge	SalePrice
YearBuilt	1.000000	0.592855	-1.000000	-0.592855	0.522897
YearRemodAdd	0.592855	1.000000	-0.592855	-1.000000	0.507101
HouseAge	-1.000000	-0.592855	1.000000	0.592855	-0.522897
RemodelAge	-0.592855	-1.000000	0.592855	1.000000	-0.507101
SalePrice	0.522897	0.507101	-0.522897	-0.507101	1.000000

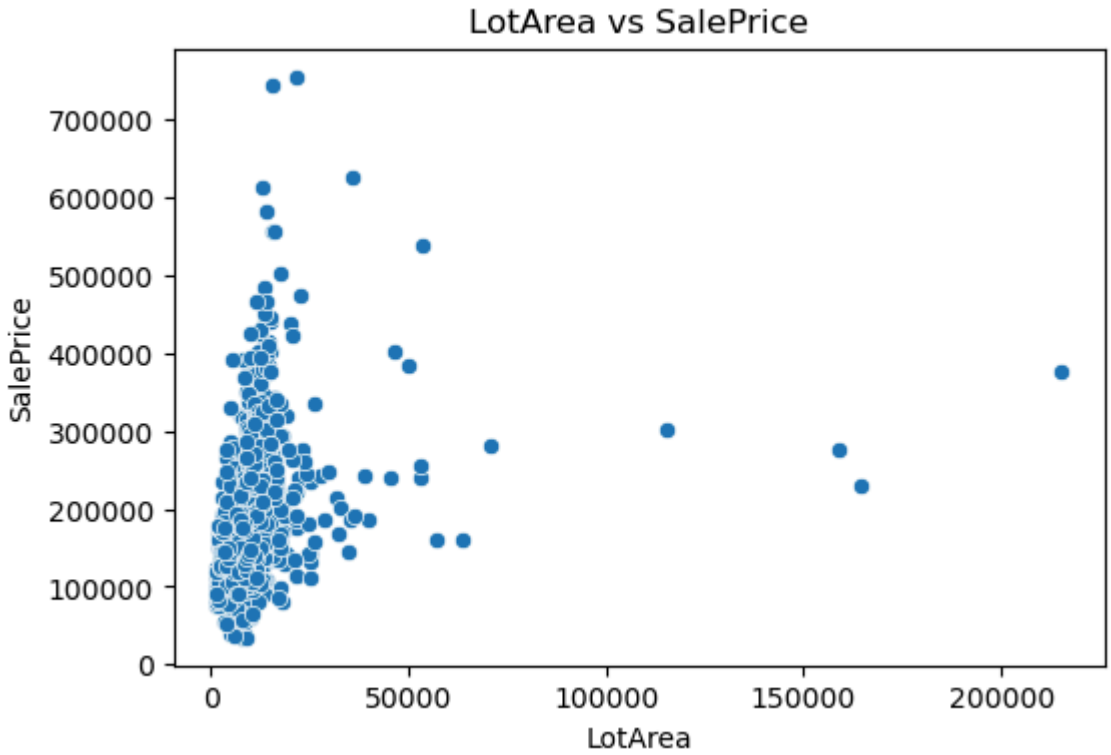


In [23]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x=train_df['OverallCond'], y=train_df['SalePrice'])
plt.title("OverallCond vs SalePrice")
plt.show()

# Mean SalePrice per condition
train_df.groupby('OverallCond')['SalePrice'].mean()
```

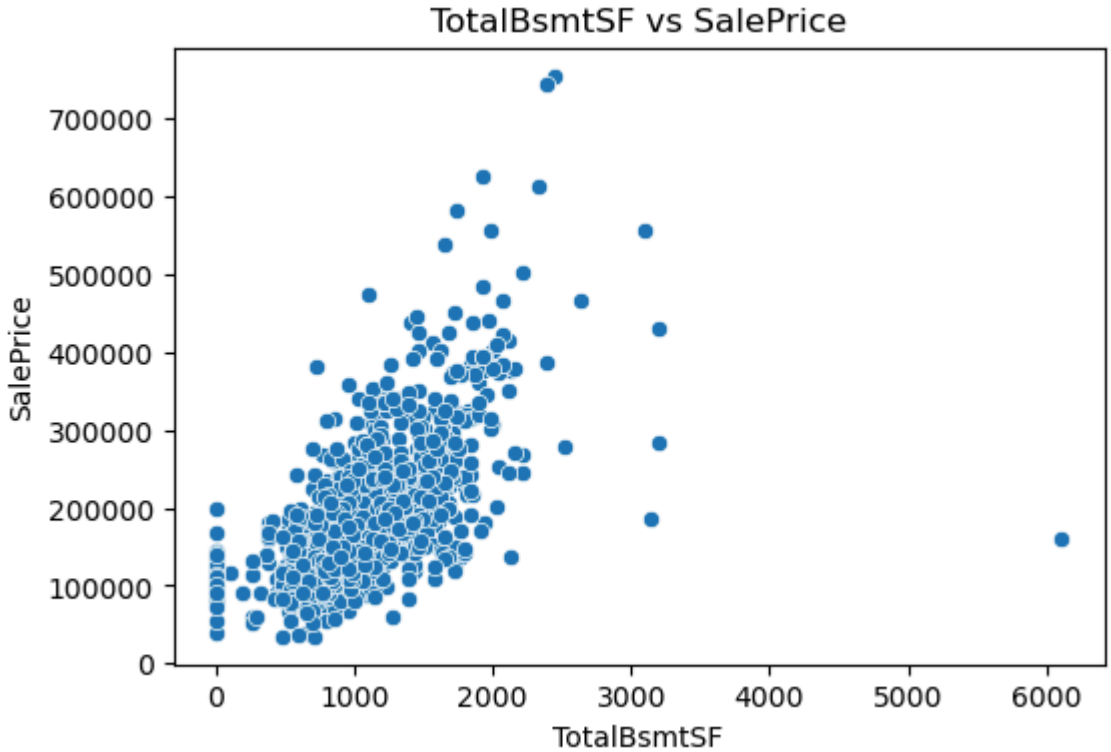


Out[23]:
```
OverallCond
1     61000.000000
2    141986.400000
3    101929.400000
4    120438.438596
5    203146.914738
6    153961.591270
7    158145.487805
8    155651.736111
9    216004.545455
Name: SalePrice, dtype: float64
```

In [24]:
```
continuous_vars = ['LotArea', 'TotalBsmtSF']

for var in continuous_vars:
    plt.figure(figsize=(6,4))
    sns.scatterplot(x=train_df[var], y=train_df['SalePrice'])
    plt.title(f"{var} vs SalePrice")
    plt.show()

    print(f"Correlation between {var} and SalePrice:",
          train_df[var].corr(train_df['SalePrice']))
```

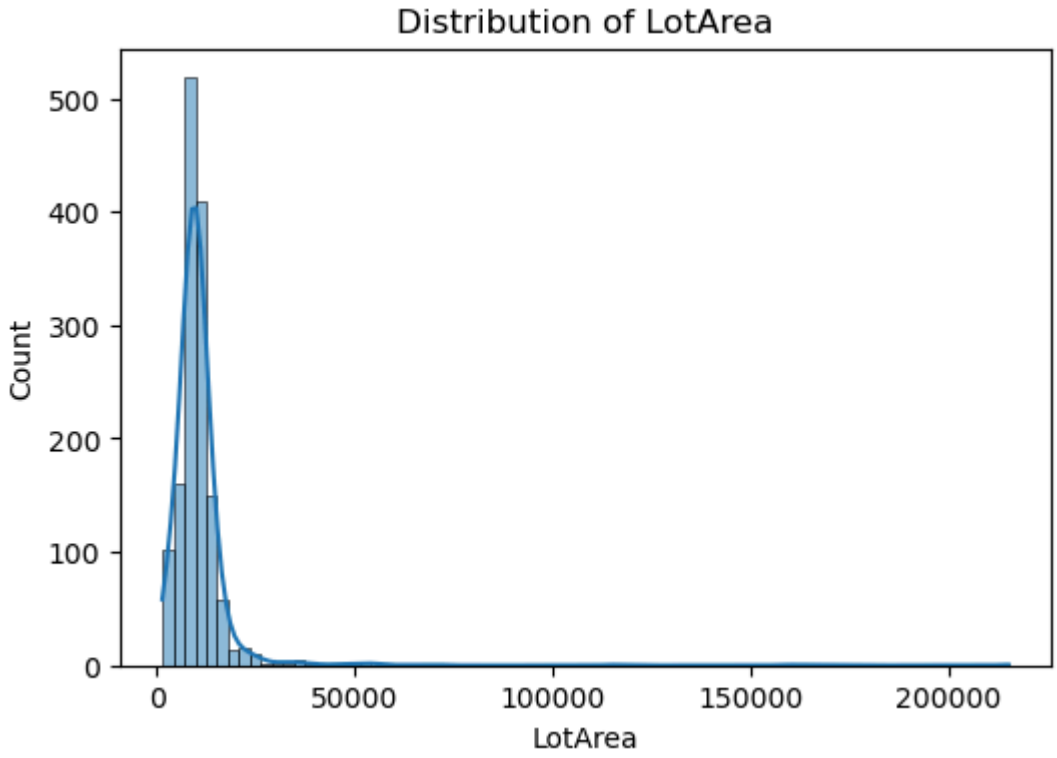


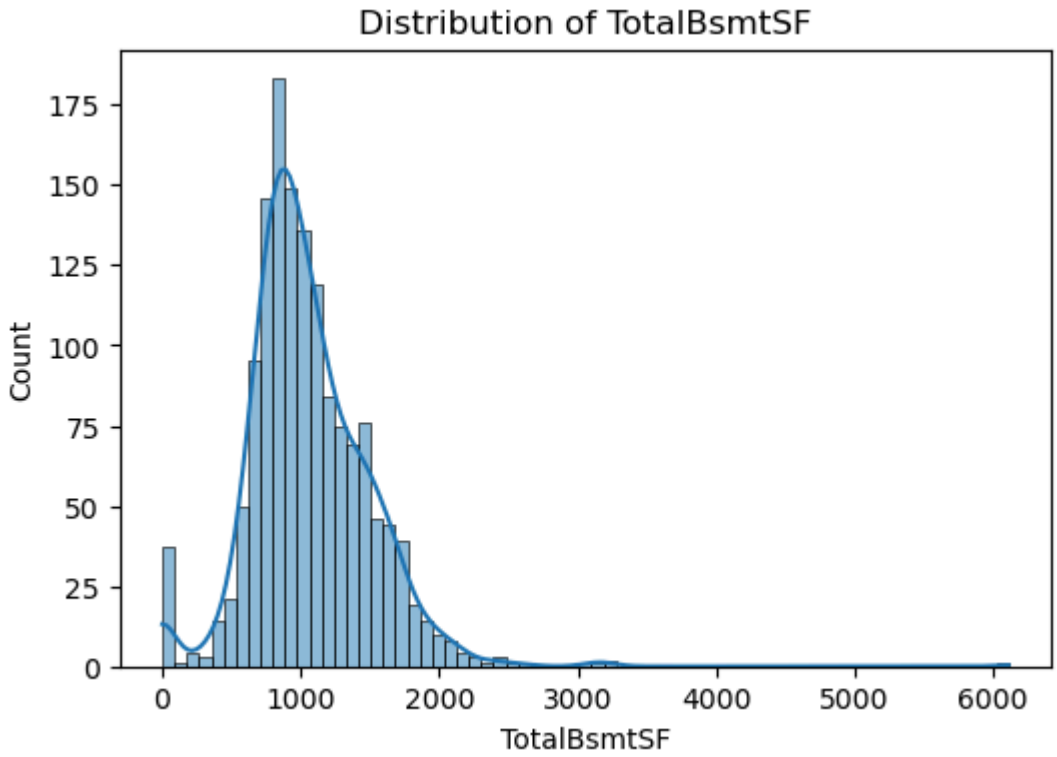

```
Correlation between LotArea and SalePrice: 0.2638433538714056
```

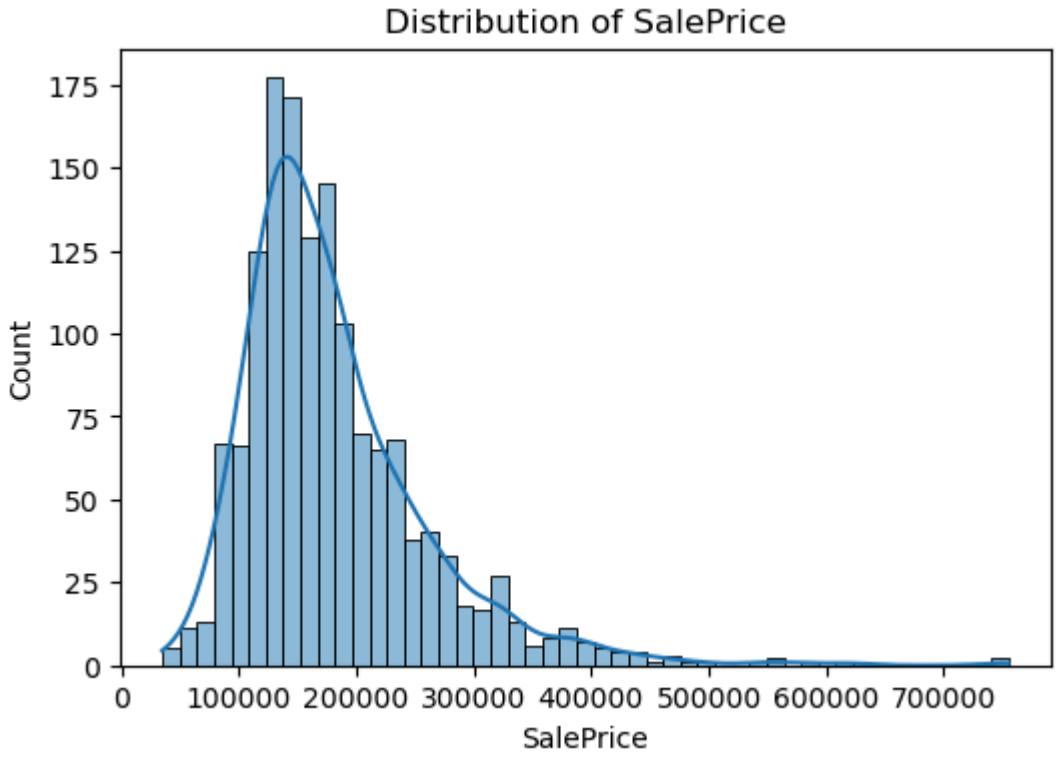



```
Correlation between TotalBsmtSF and SalePrice: 0.6135805515591953
```

In [25]:
```
continuous_features = ['LotArea', 'TotalBsmtSF', 'SalePrice']

for col in continuous_features:
    plt.figure(figsize=(6,4))
    sns.histplot(train_df[col], kde=True)
    plt.title(f"Distribution of {col}")
    plt.show()
```

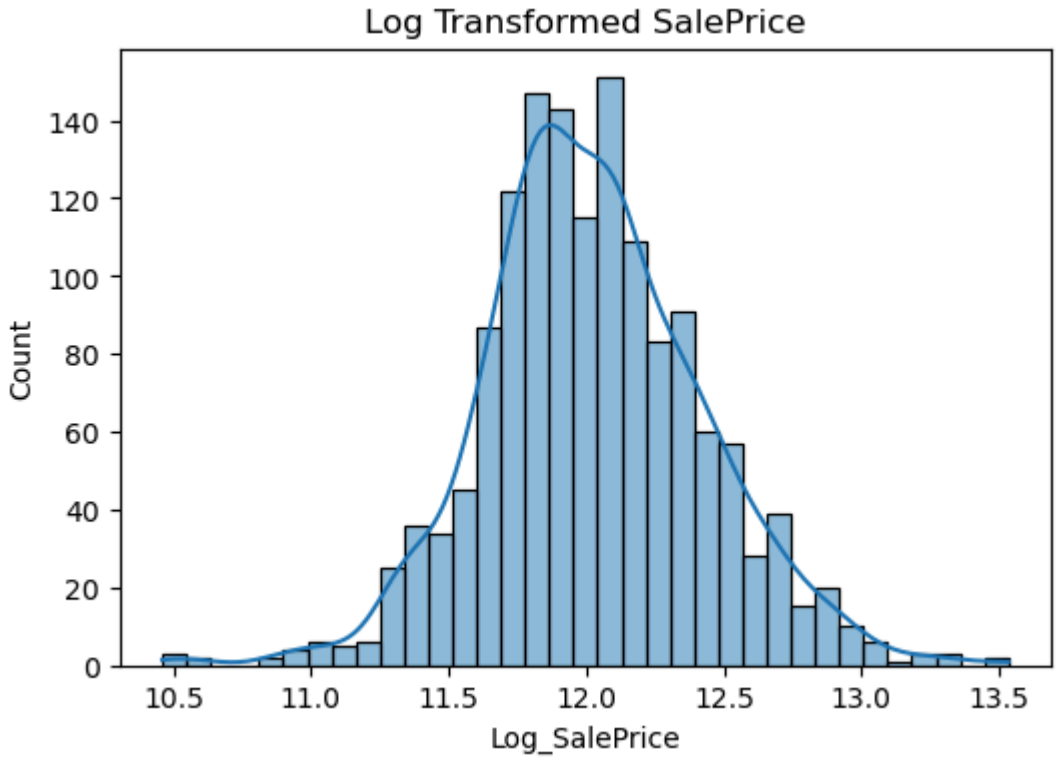

Distribution of LotArea
Count
500
400
300
200
100
0
0
50000
100000
150000
200000
LotArea


Distribution of TotalBsmtSF
Count
175
150
125
100
75
50
25
0
0
1000
2000
3000
4000
5000
6000
TotalBsmtSF



In [26]:
```
# Apply log transformation
train_df['Log_SalePrice'] = np.log1p(train_df['SalePrice'])
train_df['Log_LotArea'] = np.log1p(train_df['LotArea'])
train_df['Log_TotalBsmtSF'] = np.log1p(train_df['TotalBsmtSF'])

# Compare distribution after log
plt.figure(figsize=(6,4))
sns.histplot(train_df['Log_SalePrice'], kde=True)
plt.title("Log Transformed SalePrice")
plt.show()
```

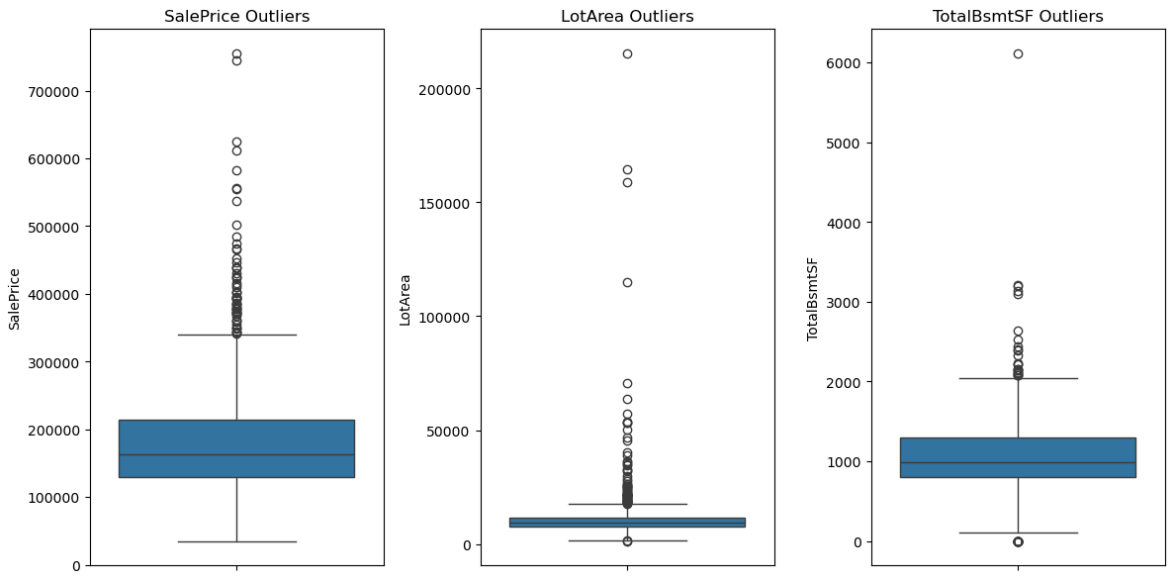
```
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\350939950.py:2: SettingWithCopy
Warning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  train_df['Log_SalePrice'] = np.log1p(train_df['SalePrice'])
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\350939950.py:3: SettingWithCopy
Warning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  train_df['Log_LotArea'] = np.log1p(train_df['LotArea'])
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\350939950.py:4: SettingWithCopy
Warning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stabl
e/user_guide/indexing.html#returning-a-view-versus-a-copy
  train_df['Log_TotalBsmtSF'] = np.log1p(train_df['TotalBsmtSF'])
```



In [28]:
```
train_df = df[df['SalePrice'].notnull()].copy()
```

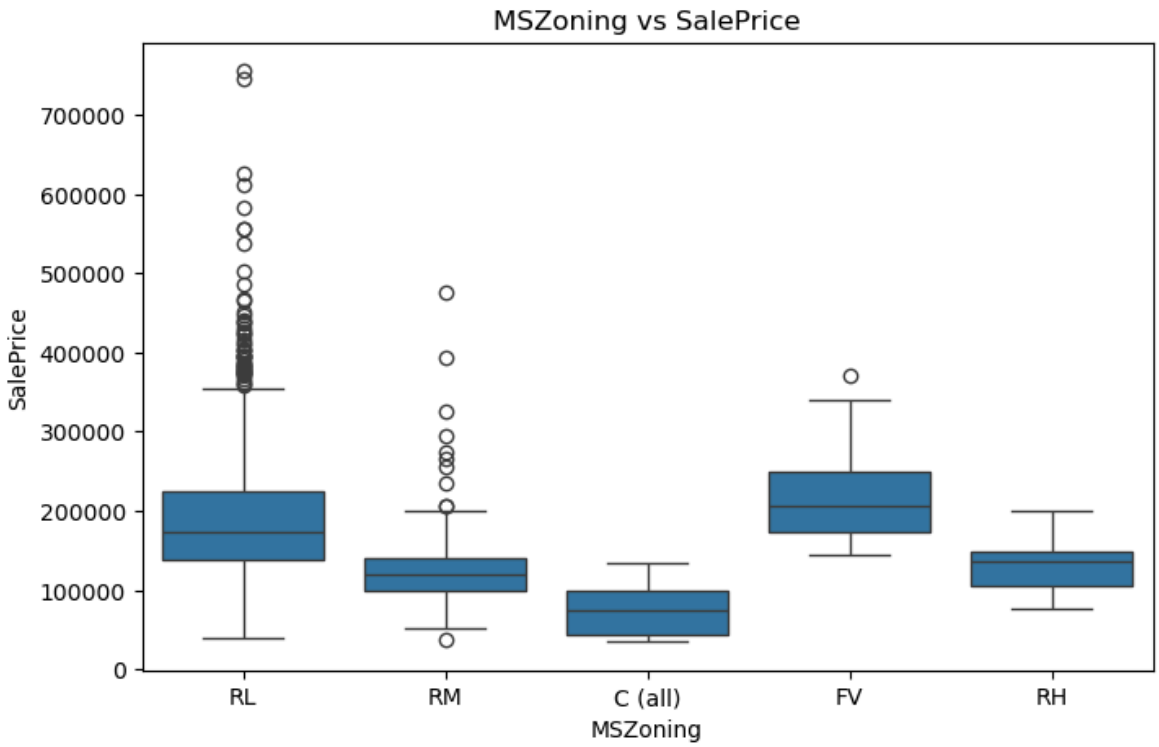
In [ ]:
```
---------------------------------------- TASK 3 ------------------------------
Q1. Find Outliers
```

In [30]:
```
plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
sns.boxplot(y=train_df['SalePrice'])
plt.title("SalePrice Outliers")

plt.subplot(1,3,2)
sns.boxplot(y=train_df['LotArea'])
plt.title("LotArea Outliers")

plt.subplot(1,3,3)
sns.boxplot(y=train_df['TotalBsmtSF'])
plt.title("TotalBsmtSF Outliers")

plt.tight_layout()
plt.show()
```



In [ ]:
```
Q2) Relationship Between Categorical Features & SalePrice
 MSZoning vs SalePrice
```

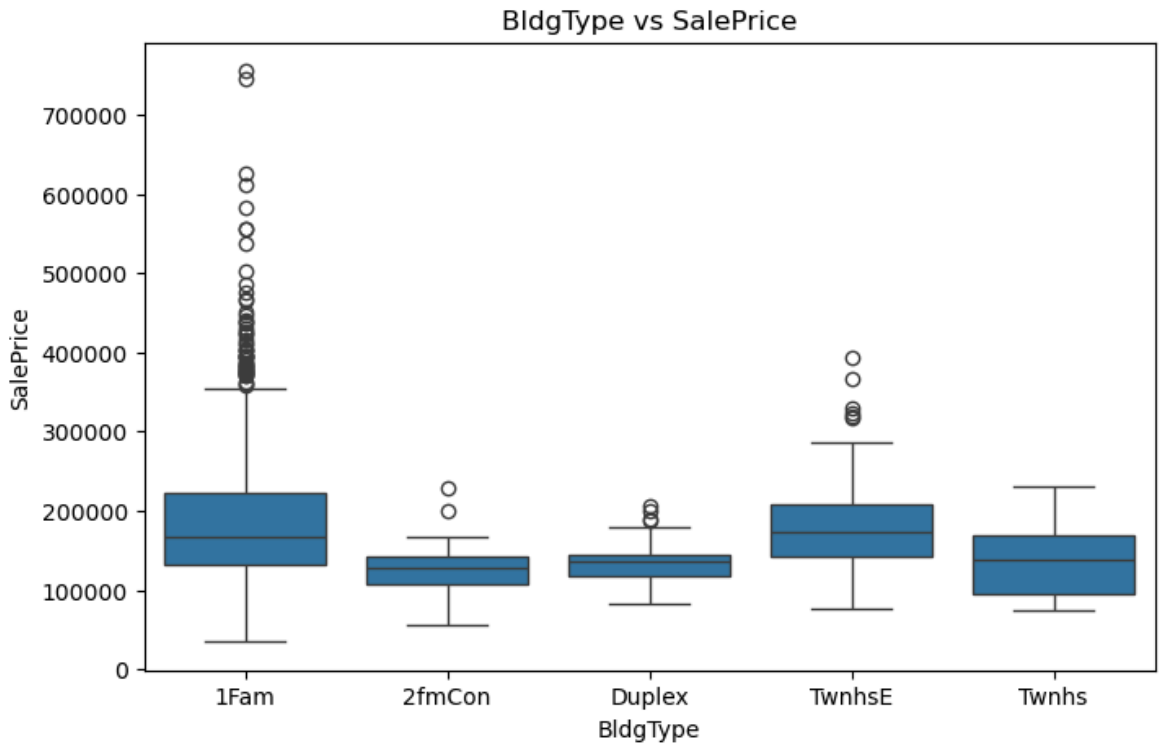
In [31]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x='MSZoning', y='SalePrice', data=train_df)
plt.title("MSZoning vs SalePrice")
plt.show()

train_df.groupby('MSZoning')['SalePrice'].mean()
```



Out[31]:
```
MSZoning
C (all)     74528.000000
FV         214014.061538
RH         131558.375000
RL         191004.994787
RM         126316.830275
Name: SalePrice, dtype: float64
```

In [ ]:
```
BldgType vs SalePrice
```

In [32]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x='BldgType', y='SalePrice', data=train_df)
plt.title("BldgType vs SalePrice")
plt.show()

train_df.groupby('BldgType')['SalePrice'].mean()
```



Out[32]:
```
BldgType
1Fam      185763.807377
2fmCon    128432.258065
Duplex    133541.076923
Twnhs     135911.627907
TwnhsE    181959.342105
Name: SalePrice, dtype: float64
```

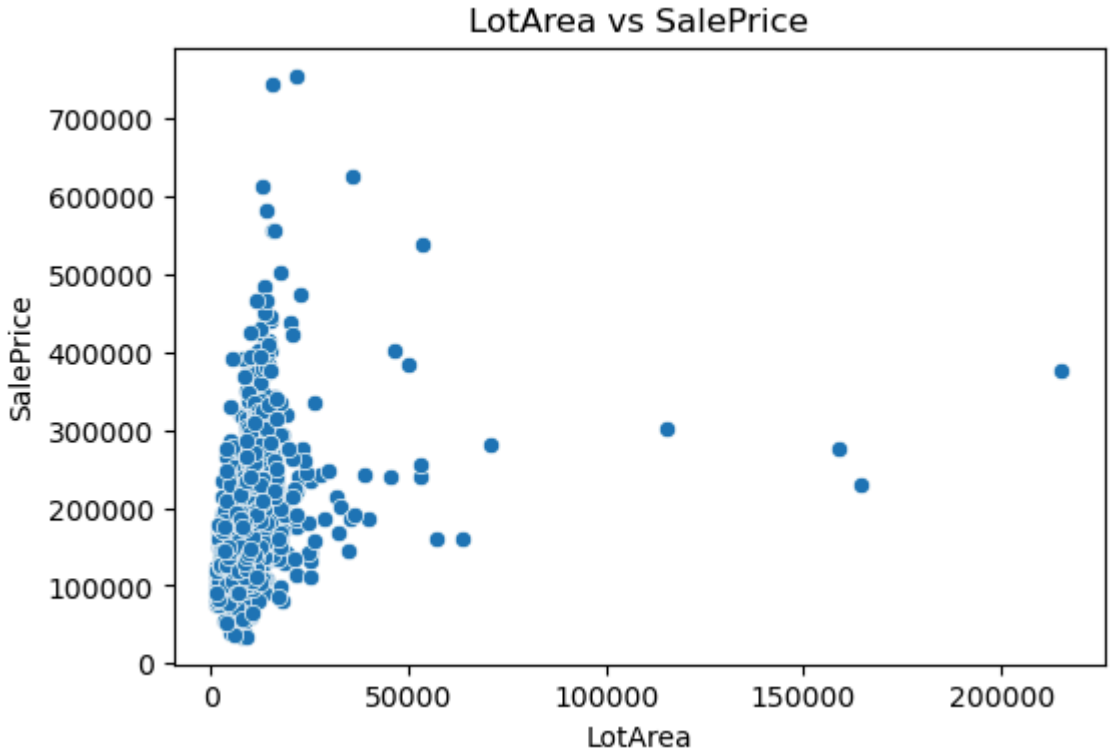
In [ ]:
```
Q3) Correlation Between Numerical Features & SalePrice
```

In [33]:
```
correlation = train_df.corr(numeric_only=True)['SalePrice'].sort_values(ascendin

print("Correlation with SalePrice:")
print(correlation)
```

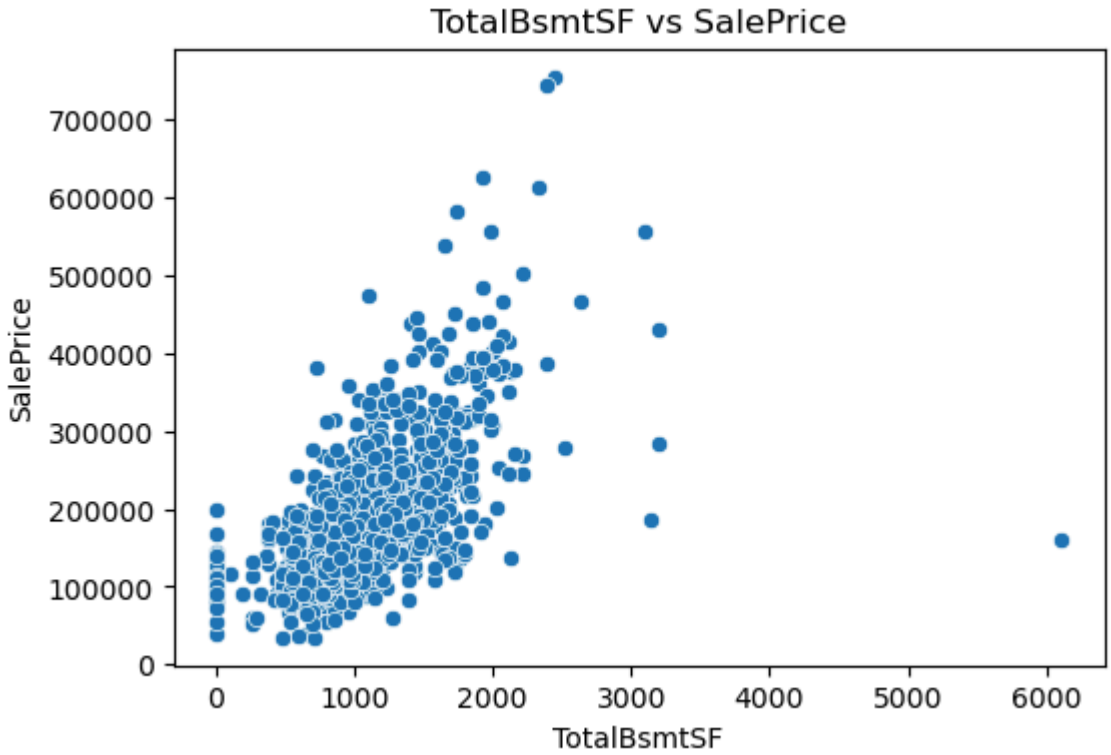
```
Correlation with SalePrice:
SalePrice       1.000000
TotalBsmtSF     0.613581
YearBuilt       0.522897
YearRemodAdd    0.507101
LotArea         0.263843
BsmtFinSF2     -0.011378
Id             -0.021917
OverallCond    -0.077856
MSSubClass     -0.084284
Name: SalePrice, dtype: float64
```

In [ ]:
```
Q4) Continuous Features vs SalePrice
```

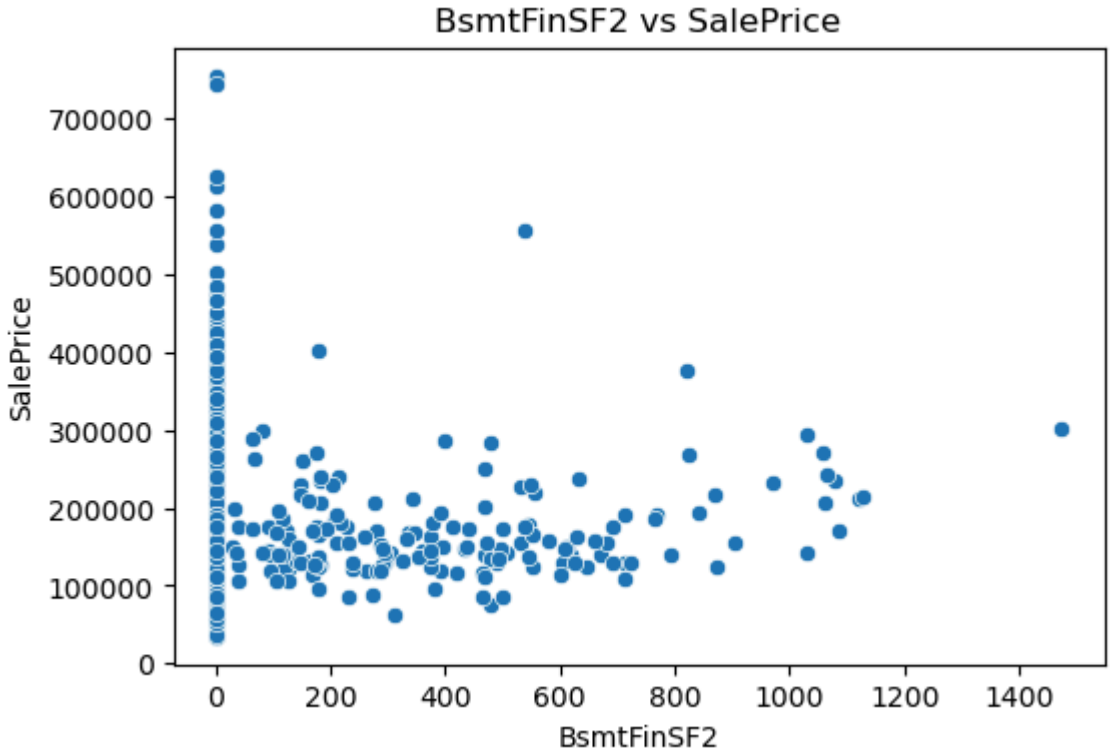
In [34]:
```
continuous_vars = ['LotArea', 'TotalBsmtSF', 'BsmtFinSF2']

for var in continuous_vars:
    plt.figure(figsize=(6,4))
    sns.scatterplot(x=train_df[var], y=train_df['SalePrice'])
    plt.title(f"{var} vs SalePrice")
    plt.show()

    print(f"Correlation ({var}):",
          train_df[var].corr(train_df['SalePrice']))
```




```
Correlation (LotArea): 0.2638433538714056
```




```
Correlation (TotalBsmtSF): 0.6135805515591953
```




```
Correlation (BsmtFinSF2): -0.011378121450215115
```

In [ ]:
```
Q5) Feature Engineering
Handling Missing Values
Numerical - Median Imputation
Categorical - Mode Imputation
```

In [35]:
```
# Handle Numerical Missing Values
num_cols = df.select_dtypes(exclude=['object']).columns

for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

```
# Handle Categorical Missing Values
cat_cols = df.select_dtypes(include=['object']).columns

for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

print("Missing values after imputation:")
print(df.isnull().sum())
```

```
Missing values after imputation:
Id               0
MSSubClass       0
MSZoning         0
LotArea          0
LotConfig        0
BldgType         0
OverallCond      0
YearBuilt        0
YearRemodAdd     0
Exterior1st      0
BsmtFinSF2       0
TotalBsmtSF      0
SalePrice        0
dtype: int64
```

```
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\2653964849.py:5: FutureWarning:
A value is trying to be set on a copy of a DataFrame or Series through chained as
signment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df[col].fillna(df[col].median(), inplace=True)
C:\Users\asmit\AppData\Local\Temp\ipykernel_23604\2653964849.py:11: FutureWarnin
g: A value is trying to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df[col].fillna(df[col].mode()[0], inplace=True)
```

In [ ]:
```
Handle Categorical Variables (One-Hot Encoding
```

In [36]:
```
df_encoded = pd.get_dummies(df, drop_first=True)

print("Shape after Encoding:", df_encoded.shape)
```

```
Shape after Encoding: (2919, 35)
```

In [ ]:
```
Handle Numerical Variables (Scaling)
```

In [38]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

num_cols_encoded = df_encoded.select_dtypes(exclude=['object']).columns

df_encoded[num_cols_encoded] = scaler.fit_transform(df_encoded[num_cols_encoded]
```

In [ ]:
```
Handle Temporal Variables (Feature Engineering)
```

In [39]:
```
current_year = 2024

df['HouseAge'] = current_year - df['YearBuilt']
df['RemodelAge'] = current_year - df['YearRemodAdd']

df[['YearBuilt','YearRemodAdd','HouseAge','RemodelAge']].head()
```

Out[39]:

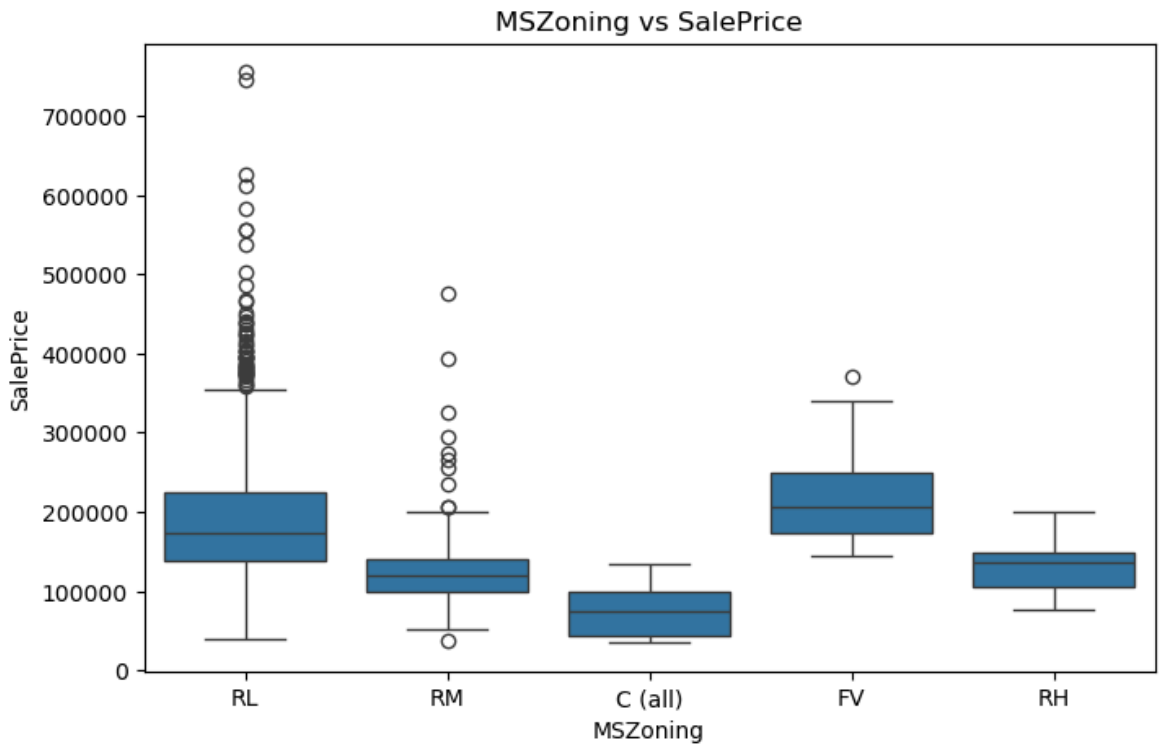
	YearBuilt	YearRemodAdd	HouseAge	RemodelAge
0	2003	2003	21	21
1	1976	1976	48	48
2	2001	2002	23	22
3	1915	1970	109	54
4	2000	2000	24	24

In [ ]:
```
---------------------------------------- Advance Level Questions---------------

Q11: How does Location (MSZoning + LotConfig) influence SalePrice?
1: Mean Price by Zoning
```

In [40]:
```
train_df.groupby('MSZoning')['SalePrice'].mean().sort_values(ascending=False)
```

Out[40]:
```
MSZoning
FV         214014.061538
RL         191004.994787
RH         131558.375000
RM         126316.830275
C (all)     74528.000000
Name: SalePrice, dtype: float64
```

In [41]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x='MSZoning', y='SalePrice', data=train_df)
plt.title("MSZoning vs SalePrice")
plt.show()
```

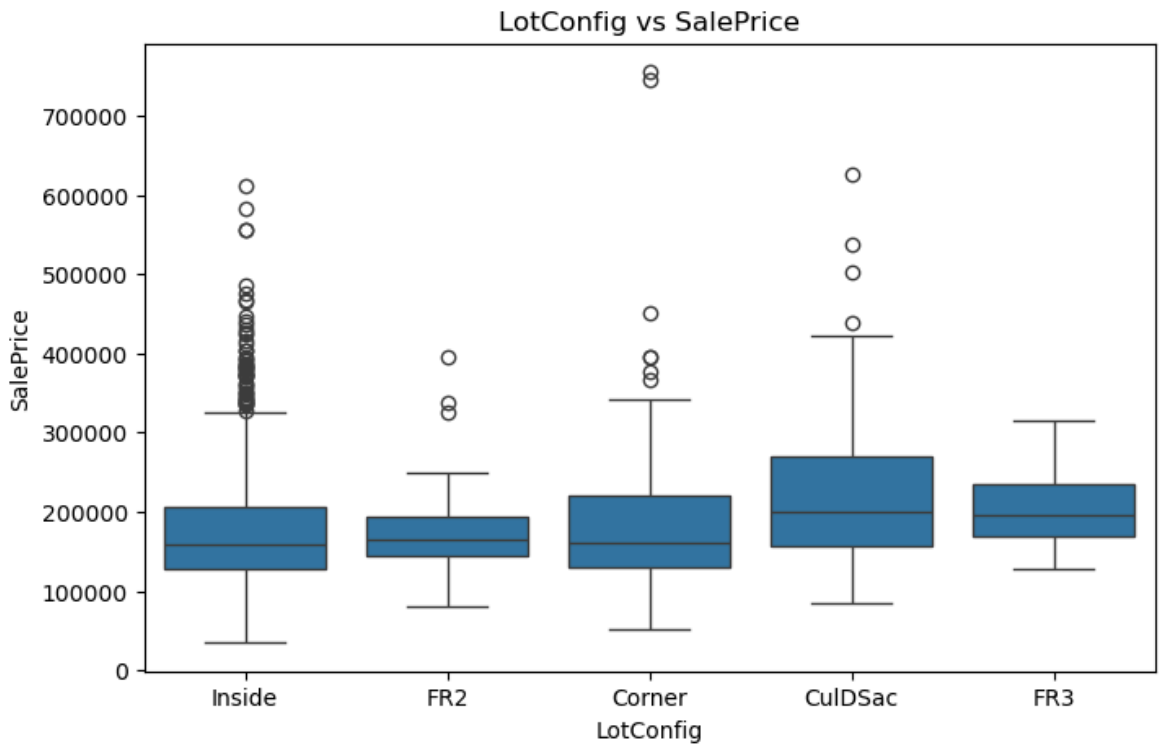


In [ ]:
```
2: Mean Price by Lot Configuration
```

In [42]:
```
train_df.groupby('LotConfig')['SalePrice'].mean().sort_values(ascending=False)
```

Out[42]:
```
LotConfig
CulDSac    223854.617021
FR3        208475.000000
Corner     181623.425856
FR2        177934.574468
Inside     176938.047529
Name: SalePrice, dtype: float64
```

In [43]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x='LotConfig', y='SalePrice', data=train_df)
plt.title("LotConfig vs SalePrice")
plt.show()
```

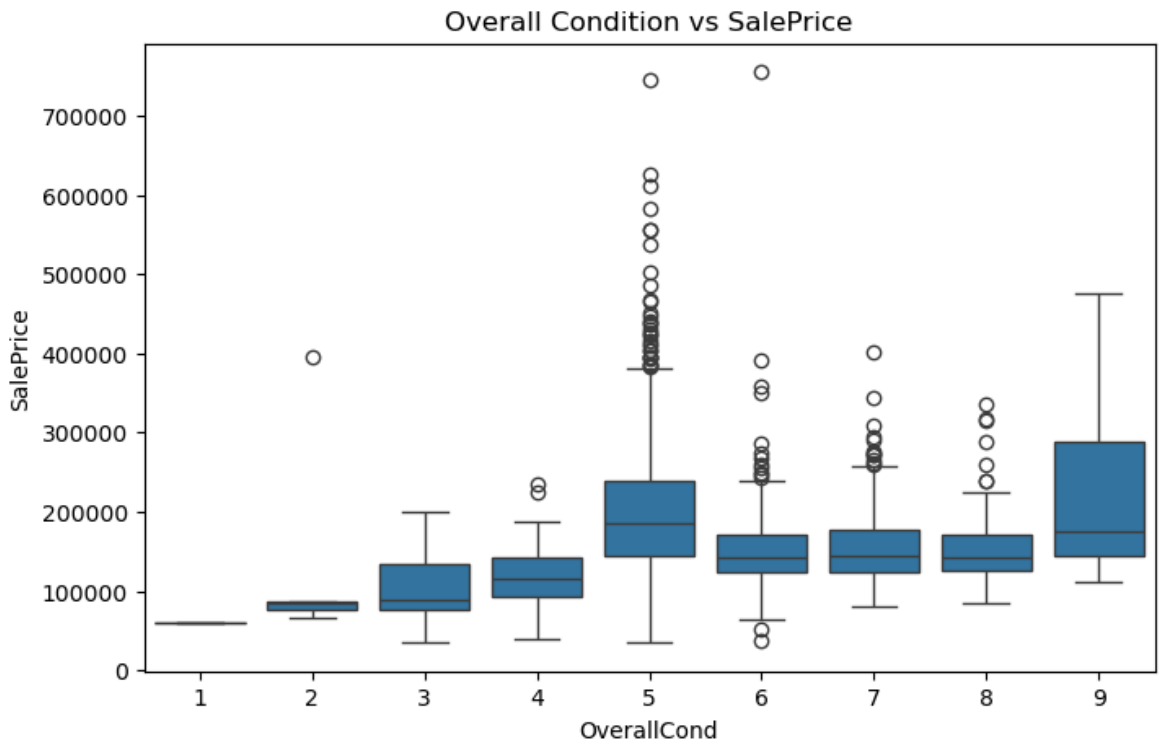


In [ ]:
```
Q12: Impact of Overall Condition on SalePrice
```

In [44]:
```
train_df.groupby('OverallCond')['SalePrice'].mean()
```

Out[44]:
```
OverallCond
1     61000.000000
2    141986.400000
3    101929.400000
4    120438.438596
5    203146.914738
6    153961.591270
7    158145.487805
8    155651.736111
9    216004.545455
Name: SalePrice, dtype: float64
```

In [45]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x='OverallCond', y='SalePrice', data=train_df)
plt.title("Overall Condition vs SalePrice")
plt.show()
```

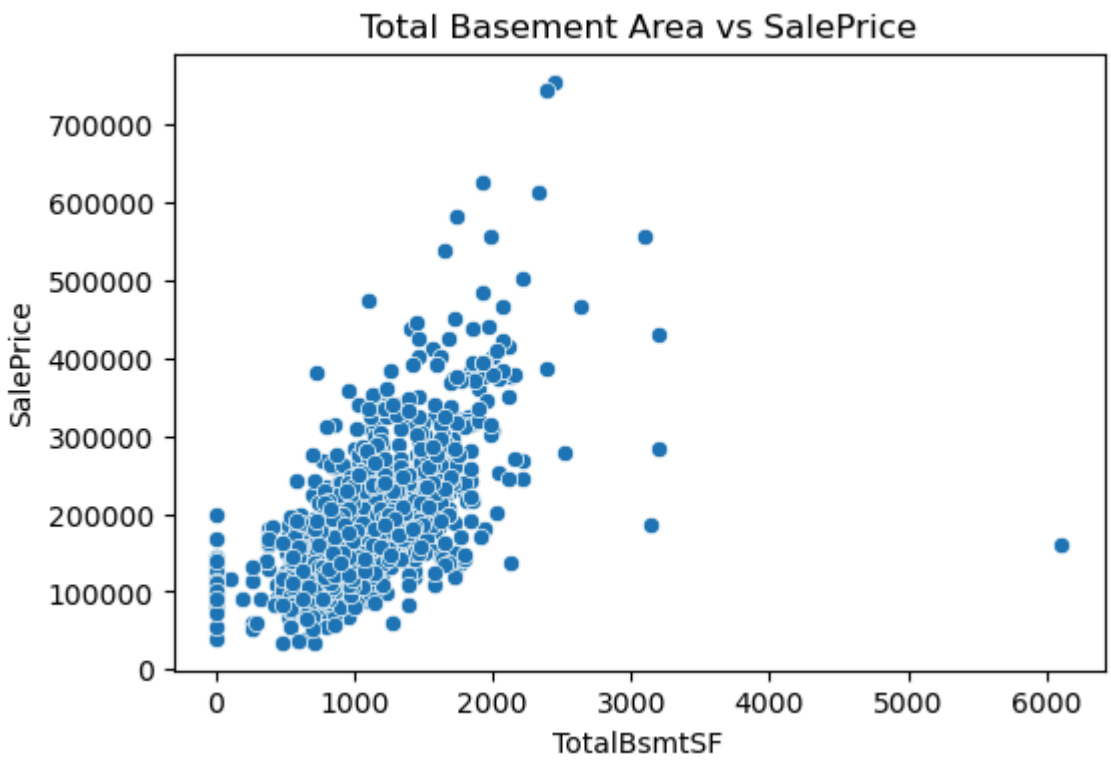


In [ ]:
```
Q13: Does Basement Size (TotalBsmtSF) Affect Price?
```

In [46]:
```
train_df['TotalBsmtSF'].corr(train_df['SalePrice'])
```

Out[46]:
```
np.float64(0.6135805515591953)
```

In [47]:
```
plt.figure(figsize=(6,4))
sns.scatterplot(x=train_df['TotalBsmtSF'], y=train_df['SalePrice'])
plt.title("Total Basement Area vs SalePrice")
plt.show()
```



In [48]:
```
train_df['HasBasement'] = train_df['TotalBsmtSF'].apply(lambda x: 1 if x > 0 els

train_df.groupby('HasBasement')['SalePrice'].mean()
```

Out[48]:
```
HasBasement
0    105652.891892
1    182878.277583
Name: SalePrice, dtype: float64
```

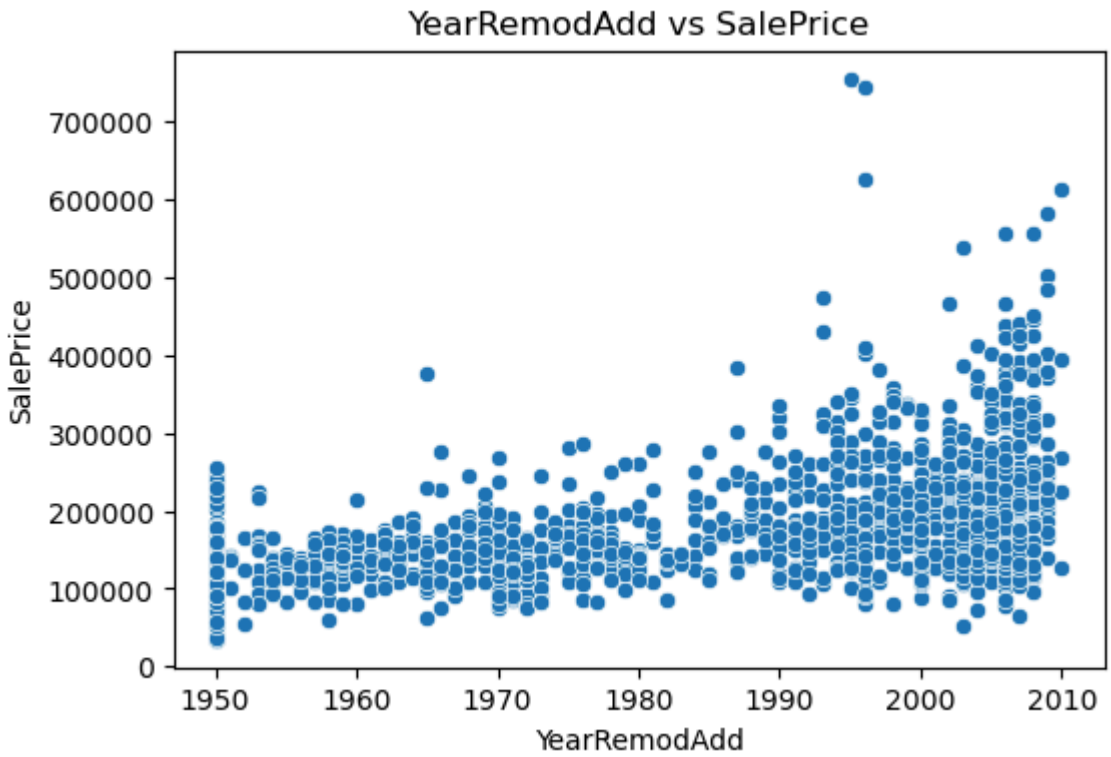
In [ ]:
```
Q14: Remodeling Impact (YearRemodAdd)
```

In [50]:
```
current_year = 2024
train_df['RemodelAge'] = current_year - train_df['YearRemodAdd']
```

In [51]:
```
train_df['RemodelAge'].corr(train_df['SalePrice'])
```

Out[51]:
```
np.float64(-0.5071009671113856)
```

In [52]:
```
plt.figure(figsize=(6,4))
sns.scatterplot(x=train_df['YearRemodAdd'], y=train_df['SalePrice'])
plt.title("YearRemodAdd vs SalePrice")
plt.show()
```



In [ ]:
```
Q15: Can We Predict Using Only Categorical Features?
```

In [53]:
```
cat_features = df.select_dtypes(include=['object']).columns

df_cat = df[cat_features]
```

In [54]:
```
df_cat_encoded = pd.get_dummies(df_cat, drop_first=True)
```

In [55]:
```
X = df_cat_encoded[df['SalePrice'].notnull()]
y = df['SalePrice'][df['SalePrice'].notnull()]
```

In [58]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

In [59]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

r2_score(y_test, predictions)
```

Out[59]:
```
0.102876112029895
```

In [ ]: